

Week06 | 课时 5 | Data Factory Runbook: 从个人脚本到团队可交接系统

本课导航

Runbook 不是事后文档，而是数据工厂的操作接口	1
这节课解决什么问题	1
参考学习时间	1
学完这一讲，你应该能做到什么	1
本课产出	2
先看故障恢复泳道图	2
1. runbook 是操作接口，不是事后文档	2
2. Runbook 最小结构	2
3. UI 与 CLI 两条路径都要有	3
4. Recovery decision tree	4
5. reason code 的价值	5
6. 交接给后续周次的字段	5
7. 已知限制必须写清	6
优秀 runbook vs 不合格 runbook	6
自检清单	7
课后最小行动	7
延伸阅读	7

Runbook 不是事后文档，而是数据工厂的操作接口

这一讲把 Week06 收口成团队能照着执行的 Data Factory Runbook。

这节课解决什么问题

数据同学 A 请假，Week08 索引前发现 2026-04-17 的 ticket fact 证据缺失。B 同学打开 runbook，可以先在 UI 里查看 asset graph 和最新 metadata，也可以用 CLI 生成 backfill dry-run plan，再跑 checks，最后把 run evidence 写入报告。这个系统才算可交接；如果只能等 A 回来，那只是个人脚本。

一个数据工厂是否可运营，不取决于作者本人能不能救火，而取决于：

另一个同学拿到 runbook 后，能不能定位、决策、重跑、验证、记录和交接。

参考学习时间

45–55 分钟

学完这一讲，你应该能做到什么

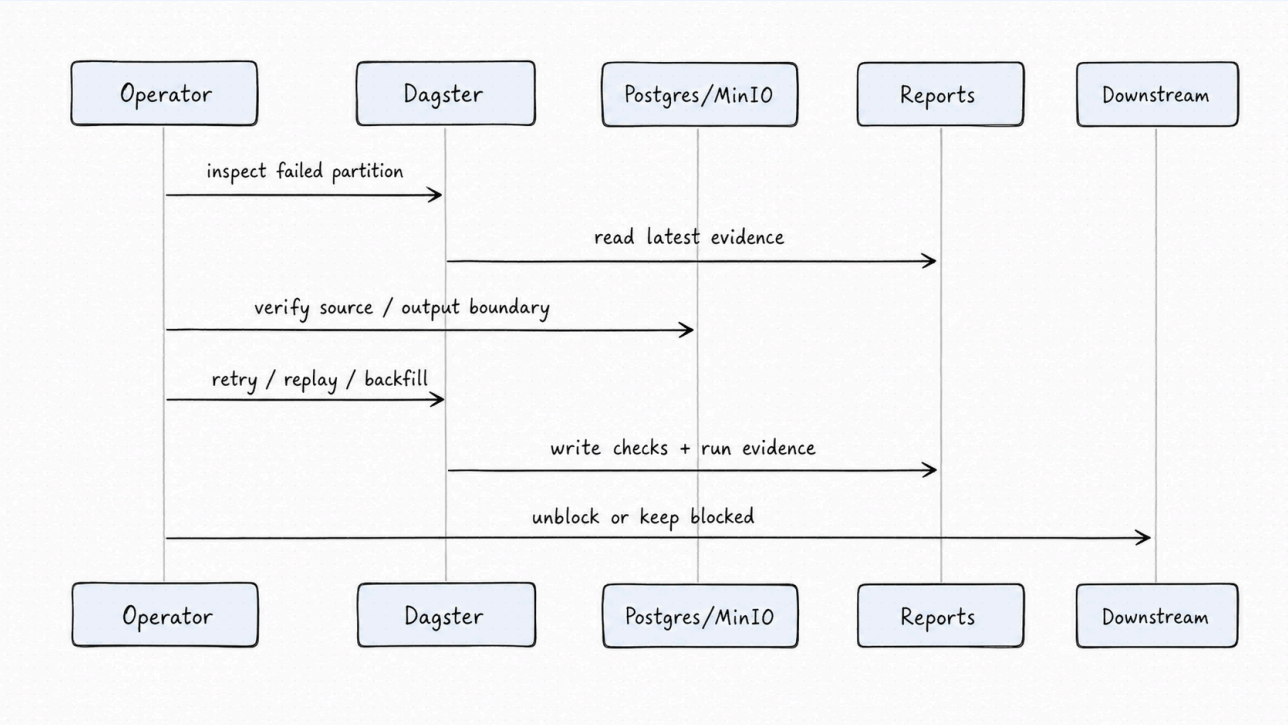
1. 写出 Week06 Data Factory Runbook 的最小结构。
2. 设计 UI 与 CLI 两条恢复路径。
3. 用 reason code 记录为什么 retry / replay / backfill。
4. 把 run evidence 交接给 Week07 / Week08 / Week11 / Week14。

5. 说明已知限制，而不是伪装生产完整。

本课产出

- [runbooks/week06-data-factory.md](#)
- [reports/week06/week06_delivery_summary.md](#)

先看故障恢复泳道图



这张图想让你看懂：runbook 不是“写给自己看的流程记录”，而是操作员、Dagster、存储、报告和下游之间的接口说明。它要让别人知道去哪看、怎么判、跑什么、期望输出是什么、失败后下一步是什么。

1. runbook 是操作接口，不是事后文档

文档型 runbook 只告诉你“大概流程是什么”。操作型 runbook 必须回答：

- 在哪里看当前状态；
- 怎么判断当前状态是否可信；
- 该跑什么命令；
- 期望输出和证据路径是什么；
- 失败后下一步动作是什么；
- 什么时候可以通知下游继续消费。

Week06 要交付的是操作型 runbook。它的目标不是把所有生产复杂性一次讲完，而是让 Student Core 的数据工厂可学、可跑、可交接。

2. Runbook 最小结构

Runbook 章节	要回答的问题	缺失后果
Scope and non-goals	本手册管什么、不管什么	误操作或抢跑 Week07 / Week08 / Week14

Runbook 章节	要回答的问题	缺失后果
Required services	需要哪些服务在线	排障方向错误
Environment variables	用哪套配置	本机和容器漂移
Load Definitions	资产图是否可加载	Dagster 入口不可靠
Observe sources	输入状态是否可用	盲目运行
Materialize core assets	如何生成核心资产	无法复现
Backfill / replay decision tree	失败后怎么选动作	一失败就全量重跑
Run checks	怎么验收资产	坏数据被下游消费
Read evidence reports	证据在哪里	无法交接和审计
Known limitations and handoff	当前不做什么	伪装生产完整

当前项目的真实 runbook 路径是：

`runbooks/week06-data-factory.md`

它覆盖 Dagster definitions 加载、daily partition、dry-run backfill、五类 checks、run evidence、Docker/ Podman 兼容性和 downstream decision。

3. UI 与 CLI 两条路径都要有

UI 适合快速定位图、状态和 metadata；CLI 适合脚本化、CI、作业提交和无 UI 环境。真实团队必须两条路径互相印证，不能只依赖截图，也不能只给一堆命令让新人猜。

场景	UI 路径	CLI 路径
看资产图	Dagster Asset Graph	<code>pytest tests/integration/test_week06_defin -q</code>
看核心资产	Week06 asset group	<code>pytest tests/integration/test_week06_asset -q</code>
补某个分区	选择 partition / materialization context	<code>python -m pipelines.data_factory.backfill_p --partition 2026-04-17 --mode dry-run</code>
看检查结果	Asset Checks tab	<code>pytest tests/integration/test_week06_asset_checks.py -q</code>
看证据	Asset materialization meta-data	<code>reports/week06/run_evidence/*.json</code>
交付总结	delivery summary asset	<code>reports/week06/week06_delivery_summary.md</code>

真实课堂命令以项目 runbook 为准：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    pytest tests/contract/test_week06_run_evidence_schema.py -q
```

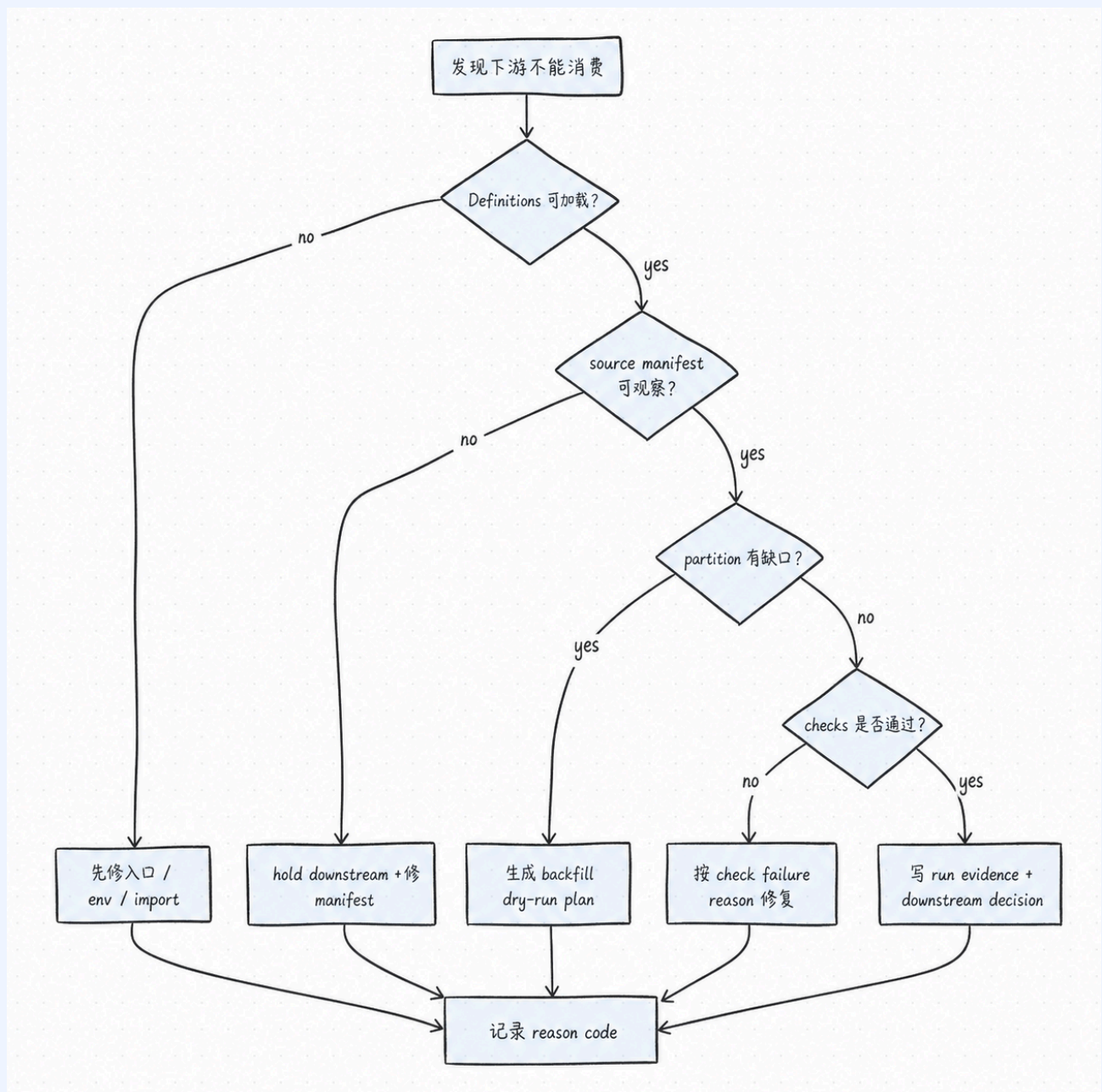
```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    pytest tests/integration/test_week06_definitions_loadable.py -q
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    python -m pipelines.data_factory.backfill_plan --partition 2026-04-17 --mode dry-run
```

```
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    pytest tests/integration/test_week06_asset_checks.py tests/integration/
test_week06_run_evidence_generation.py -q
```

验收点：UI 能帮助定位，但最终交接要落到 `reports/week06/` 和 `runbooks/week06-data-factory.md`。

4. Recovery decision tree



决策树不是为了自动化所有判断，而是避免靠感觉救火。先确认入口能加载，再看输入声明，再判断 partition gap，再跑 checks，最后才写 evidence 和 downstream decision。

⚠️ 一失败就全量重跑，是 runbook 没有发挥作用的信号

runbook 的价值是把“我觉得应该重跑”改成“因为这个 reason code，所以选择这个恢复动作，并留下这份 evidence”。

5. reason code 的价值

reason code 让故障判断可以被复核、统计和交接。下面的表同时包含项目当前已落地的 reason code，以及课程里建议映射的操作含义。

reason code	含义	后续动作
structured_manifest_missing_or_empty	输入声明缺失或资产为空	修 manifest / hold downstream
partition_has_no_seed_rows	选定分区没有 source rows	生成 backfill dry-run plan 或换正确 partition
duplicate_ticket_id	幂等失败或输入重复	查 dedupe key / idempotency guard
required_field_nulls_detected	必填字段缺失	block / quarantine / 修输入
no_valid_input_rows	ingest wrapper 没有有效输入	review manifest 和 seed data
week04_lakehouse_not_available	Week04 optional 依赖未启用	记录 not_available，不要 fake passed
week05_analytics_not_available	Week05 optional 依赖未启用	记录 not_available，不要写 success
dry_run_no_db_write	只生成计划或 dry-run 证据	downstream_decision=dry_run_only，不放行真实下游
asset_checks_failed	至少一个 core check 失败	hold_downstream，按 check reason 修复

Prompt 里的 manifest_not_found、partition_gap_detected、optional_dependency_not_available 可以作为教学口径理解；在当前项目里，具体落地名称以上表和 pipelines/data_factory/checks.py、assets.py 为准。

6. 交接给后续周次的字段

后续周	会消费 Week06 什么	为什么
Week07	run evidence / source manifest / partition status	判断哪些文档或工单可进入解析
Week08	checks summary / downstream decision	判断是否可以索引
Week11	evidence + git_sha + data_release_id	评测绑定输入状态

后续周	会消费 Week06 什么	为什么
Week14	<code>run_id</code> / <code>asset_key</code> / <code>lineage-ready</code> 字段	治理、追责、回滚

当前项目生成的交接摘要：

`reports/week06/week06_delivery_summary.md`

它至少应该告诉下游：本次 `partition`、`status`、`downstream decision`、`run evidence` 路径和 `reason codes`。没有这些字段，下游只能相信口头说明。

7. 已知限制必须写清

! Student Core 的目标是可学、可跑、可交接，不是假装生产完整

完整 OpenLineage、lakeFS、multi-partition backfill、跨系统 lineage 平台不是 Week06 硬交付。Week06 只把 Week03–Week05 的核心链路组织成可依赖、可分区、可回填、可检查、可追溯的本地数据工厂 v1。

Runbook 里必须承认：

- Week04 lakehouse 未完成时，`snapshot` 字段可以为空；
- Week05 analytics 未完成时，`semantic/mart` 状态可以 `not_available`；
- Student Core 不依赖 Dagster+；
- OpenLineage / lakeFS / OpenMetadata 留给 Week14；
- Week07 解析、Week08 索引不在 Week06 实现；
- Week06 默认 `dry-run`，不执行破坏性历史重写。

优秀 runbook vs 不合格 runbook

维度	优秀 runbook	不合格 runbook
前置条件	写清服务、env、默认 <code>partition</code> 、可选依赖状态	只说“先启动环境”
故障判断	用 <code>reason code</code> 区分缺数据、schema、重复、optional unavailable	统一写“重跑一下”
操作路径	UI 路径和 CLI 路径都能走	只贴截图或只贴命令
验收证据	指向 <code>checks</code> 、 <code>run evidence</code> 、 <code>delivery summary</code>	只有日志片段
交接	写清 <code>downstream unblock</code> / hold 决策	作者本人能懂，别人无法接手

! 核心判断

没有 runbook 的数据工厂不可交接；不能交接，就不能算工程完成。

自检清单

- ☐ runbook 能让别人启动、观察、materialize、backfill、检查和读 evidence。
- ☐ UI / CLI 两条路径都能走，并能互相印证。
- ☐ recovery decision tree 能区分 retry / replay / backfill / restore。
- ☐ optional 依赖的限制写清楚，没有 fake passed。
- ☐ evidence 能交给 Week07 / Week08 / Week11 / Week14。
- ☐ 没有把 Week06 写成 UI 点击教程。

课后最小行动

为你的 Week06 runbook 补一个 operator checklist:

```
## Operator checklist
```

- [] 当前失败 partition:
- [] 已读取 latest evidence:
- [] 已确认 source / output boundary:
- [] Definitions 是否可加载:
- [] 选择动作: retry / rerun / replay / backfill / restore
- [] reason code:
- [] checks 结果:
- [] downstream decision:
- [] evidence/report path:

延伸阅读

- Dagster: [Backfilling data](#)¹
- Dagster: [Assets](#)²
- Dagster: [Asset checks](#)³
- OpenLineage: [Object model](#)⁴
- 项目 runbook: [runbooks/week06-data-factory.md](#)

¹Dagster backfill 能把分区恢复动作组织成可追踪操作；Week06 只使用 dry-run planning 作为 Student Core 的安全版本。

²Dagster asset graph 给 runbook 提供“看哪里、修哪里、交接什么”的资产边界，不是单纯的任务流程图。

³Asset checks 是 runbook 判断是否放行下游的依据之一；它们不能替代 run evidence，也不能替代 operator 的 downstream decision。

⁴OpenLineage object model 是后续治理、追责、回滚和发布系统的标准化方向；Week06 只预留 run_id、asset_key、git_sha、trace_id 等字段，不把 OpenLineage 做成本周硬依赖。